Smart Contract Audit Report

Contract: vulnerable_contract

Date: 2025-07-08 23:06:35

Risk Summary

Risk Score: 60/100

Risk Level: High

Vulnerabilities Found: 6

Vulnerability Findings

1. Use of system() call

Severity: Unknown

Location: system()

Description: Use of system() allows arbitrary command execution. Avoid using it in smart contracts.

2. Unchecked arithmetic operation

Severity: Unknown

Location: balance += amount

Description: Operation 'balance += amount' might cause overflow/underflow. Consider using safe math checks.

3. Unchecked arithmetic operation

Severity: Unknown

Location: balance -= amount

Description: Operation 'balance -= amount' might cause overflow/underflow. Consider using safe math checks.

4. Unchecked arithmetic operation

Severity: Unknown

Location: char* input

Description: Operation 'char* input' might cause overflow/underflow. Consider using safe math checks.

5. Unchecked arithmetic operation

Severity: Unknown

Location: a * b

Description: Operation 'a * b' might cause overflow/underflow. Consider using safe math checks.

6. Unchecked arithmetic operation

Severity: Unknown

Location: rm -rf

Description: Operation 'rm -rf' might cause overflow/underflow. Consider using safe math checks.

AI-Powered Analysis

Audit Summary: Vulnerable Contract

The vulnerable_contract smart contract has been identified to contain several critical and medium-severity vulnerabilities that can be exploited by malicious actors, leading to unintended behavior, financial losses, and reputational damage.

1. Use of system() call (High Severity)

* **Description:** The contract uses the `system()` function, which allows arbitrary command execution. This is a critical vulnerability that can be exploited to execute malicious system commands, compromising the entire system.
* **Impact:** High. An attacker can execute arbitrary system commands, including those that can drain the contract's funds, steal sensitive information, or disrupt the underlying system.
* **Exploit Path:** An attacker can inject malicious system commands through the `system()` function, allowing them to take control of the contract and the underlying system.
* **Mitigation Strategy:** Avoid using the `system()` function in smart contracts altogether. Instead, use contract-specific functionality or libraries that provide secure and isolated execution environments.

Advice for Developers: Never use the `system()` function in smart contracts, as it can lead to catastrophic consequences. Instead, focus on using contract-specific functionality and libraries that provide secure and isolated execution environments.

2. Unchecked Arithmetic Operations (Medium Severity)

* **Description:** The contract contains multiple unchecked arithmetic operations, including `balance += amount`, `balance -= amount`, `char* input`, `a * b`, and `rm -rf`. These operations can cause overflow or

underflow, leading to unintended behavior and financial losses.
* **Impact:** Medium. Unchecked arithmetic operations can lead to unintended behavior, financial losses, and reputational damage.
* **Exploit Path:** An attacker can exploit these operations to manipulate the contract's state, leading to unintended behavior and financial losses.
* **Mitigation Strategy:** Use safe math libraries and checks to prevent overflow and underflow in arithmetic operations. Implement robust input validation and sanitization to prevent malicious input.

Advice for Developers: Always use safe math libraries and checks to prevent overflow and underflow in arithmetic operations. Implement robust input validation and sanitization to prevent malicious input. Consider using libraries like OpenZeppelin's SafeMath to ensure safe arithmetic operations.

Recommendations:

1. Refactor the contract to remove the `system()` function and replace it with contract-specific functionality or libraries.
2. Implement safe math libraries and checks to prevent overflow and underflow in arithmetic operations.
3. Perform robust input validation and sanitization to prevent malicious input.
4. Conduct thorough testing and code reviews to identify and address potential vulnerabilities.
5. Consider using security auditing tools and services to identify and address vulnerabilities in the contract.

By addressing these vulnerabilities and implementing the recommended mitigation strategies, the vulnerable_contract can be made more secure, reliable, and resistant to potential exploits.